CPE477/ECG677
Embedded Security and Machine Learning

Design Assignment 2

Decision Tree Model Comparison for Activity Recognition

DO NOT REMOVE THIS PAGE DURING SUBMISSION:

Name: Carlos Funes

Group-Partner Name: _Jabril O. Smith

Email: funesc2@unlv.nevada.edu

Github Repository link (root): https://github.com/CaFu0320/senna/tree/main/DA2%20ECG677

Youtube Playlist link (root): https://youtube.com/playlist?list=PLLIny5qXi1AM&si=9r4MMgPdy5MrUBvA

Follow the submission guideline to be awarded points for this Assignment.

Submit the following for all Assignments:

1. Include modified or added code for each task and highlight the key implementations.
2. Create a private GitHub repository with a random name. Omit course identifiers and student names from the repository name.
3. Organize Python source in /src, trained JSON models in /models, input CSVs in /data, and analysis and results in /docs.
4. Include the Word report, scripts, trained models, datasets, and analysis in the repository.
5. Submit the report as a PDF in Canvas. Match the PDF filename stem to the assignment directory name in GitHub.
6. Use the playlist name “EMBSEC&ML DA2” for the demonstrations covering Tasks 1–6.
7. Include the PDF report in the repository and submit the PDF in Canvas.

Declaration

I understand the Student Academic Misconduct Policy - http://studentconduct.unlv.edu/misconduct/policy.html

“This assignment submission is my own, original work”.

Name of the Student: Carlos Funes

Decision Tree Activity Recognition

1 Abstract

We trained four decision trees to recognize curls, lateral raises, and left punches recorded with the LSM6DSV16X on the X-NUCLEO-IKS4A1 and NUCLEO-F401RE setup. The models compare raw IMU axes, raw-axis window statistics, logged quaternion components, and selected quaternion window statistics. On held-out portions of the recordings, RAW reached 97.83% and QUAT reached 87.22%, while RAW+FE and QUAT+FE classified all 59 and 81 test windows correctly, respectively. QUAT+FE used three leaves and was the smallest tree by leaf count. In Task 7, an LSM6DSO16IS ISPU implementation uses relative quaternion motion features. The retrained tree classified 102 of 105 held-out windows correctly, and the desktop C replay agreed with the reference on all 615 checked windows.

2 Schematics

The expansion board plugs into the Nucleo Arduino UNO R3 headers. The STM32 reads the selected IMU over I2C1 and communicates with MEMS Studio through USART2 and the ST-LINK virtual COM port. The two sensor targets share the physical board stack. [1, 2]

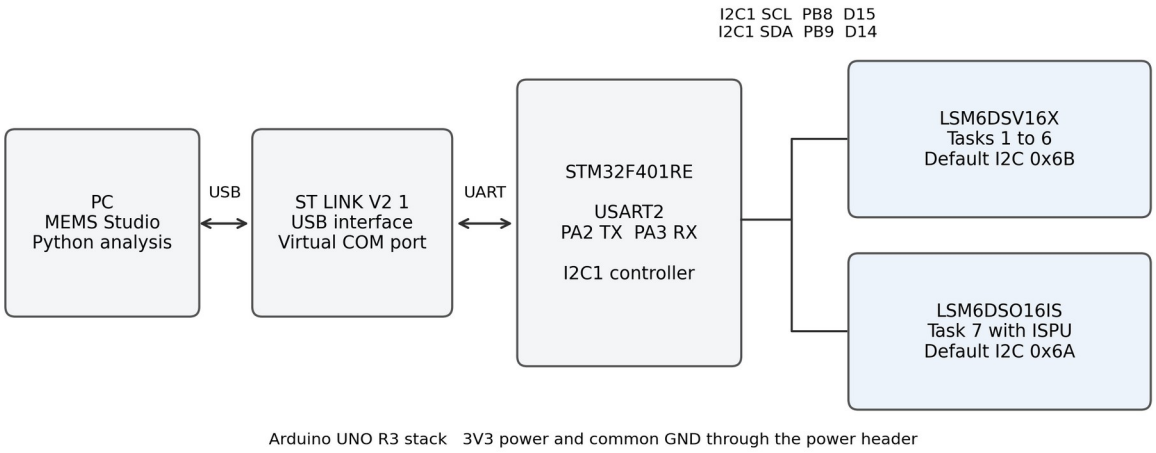


Figure 1. Hardware connections and sensor targets used across the assignment. Addresses are the board defaults documented by ST. [1, 2]

Table 1. Signals used by the hardware setup

Connection	Pin or interface	Function
I2C1 clock	PB8 / Arduino D15	STM32 to shield SCL
I2C1 data	PB9 / Arduino D14	Bidirectional sensor data
USART2	PA2 TX and PA3 RX	ST-LINK virtual COM port
Power and reference	3V3 and GND header pins	Shield power and common ground
USB	PC to ST-LINK mini-B connector	Board supply and host data link

3 Task Results

Task 1 Data Preparation and Exploratory Analysis

We recorded curls, lateral raises, and left punches in MEMS Studio using the board shown in Section 2. Each activity has one raw six-axis IMU recording and one separate quaternion recording. The raw logs contain time, three acceleration axes in mg, and three angular-rate axes in mdps. Task 1 converts them to g and degrees per second for analysis and computes offline Madgwick orientation for the raw exploratory plots. Tasks 4 and 5 use the separately logged quaternion takes in their original q0–q3 column order.

The largest observed axis magnitudes were 2.259 g and 279.16°/s. For repeat captures, we would select ±4 g and ±500°/s for headroom and choose an ODR that sustains about 100 logged samples/s to keep the two streams comparable. These raw logs delivered 71.5–72.9 samples/s during active capture, and the quaternion logs delivered about 100 samples/s. The programmed full-scale settings and sensor ODR require confirmation from the original acquisition configuration.

The quaternion parser keeps the first copy of each duplicated component column and discards zero-norm rows. The raw left-punch recording has a 246.42 s timestamp jump. Samples on both sides remain in the dataset. Frequency analysis interpolates only within continuous segments, and feature windows skip the discontinuity.

Table 2. Task 1 class-stratified 70/15/15 split

Stream	Activity	Train	Validation	Test	Total
Raw IMU	Curl	1,985	426	425	2,836
Raw IMU	Lateral raise	1,994	427	428	2,849
Raw IMU	Left punch	2,239	479	480	3,198
Raw IMU	Total	6,218	1,332	1,333	8,883
Logged quat.	Curl	2,831	606	607	4,044
Logged quat.	Lateral raise	2,835	608	607	4,050
Logged quat.	Left punch	2,835	608	608	4,051
Logged quat.	Total	8,501	1,822	1,822	12,145

The reproducible Task 1 split uses seed 42. Tasks 2–5 use chronological 70/15/15 portions of each take, so their counts differ slightly from this exploratory split. The model windows are formed after splitting.

Source and outputs are in src/data_preparation.py, data/processed, and docs/task1. Appendix A shows the split implementation.

Task 1 Signal Examples and Class Distribution

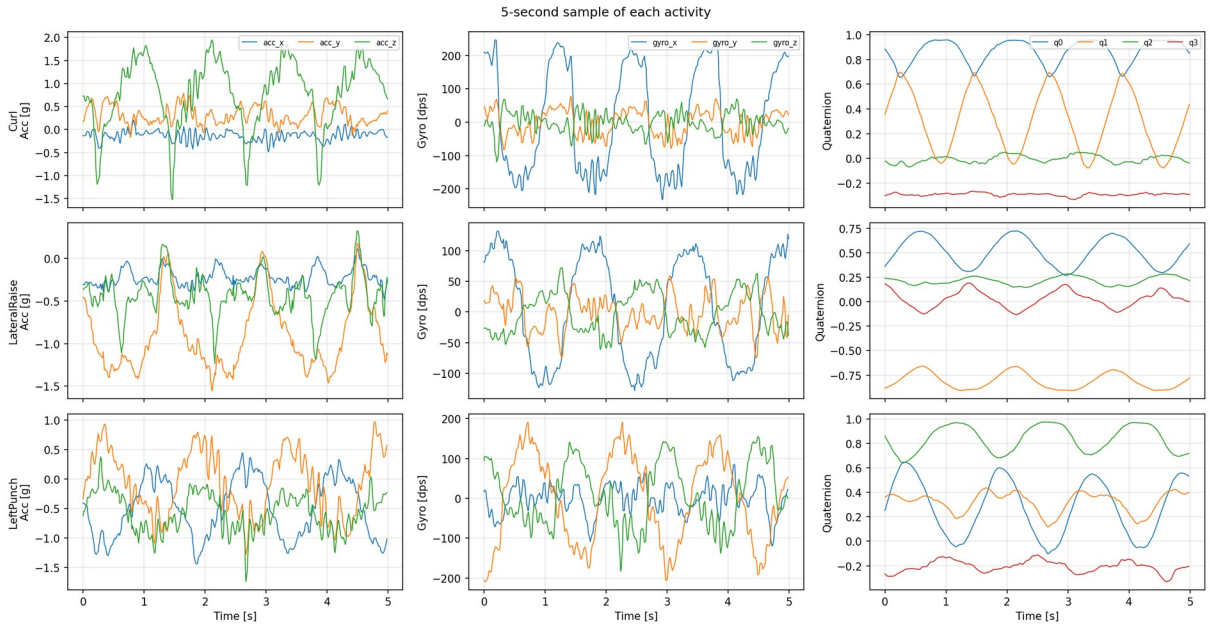


Figure 2. Short time-series excerpts for the three exercises. Acceleration and angular rate come from the raw IMU logs. The quaternion column in this plot uses offline fusion.

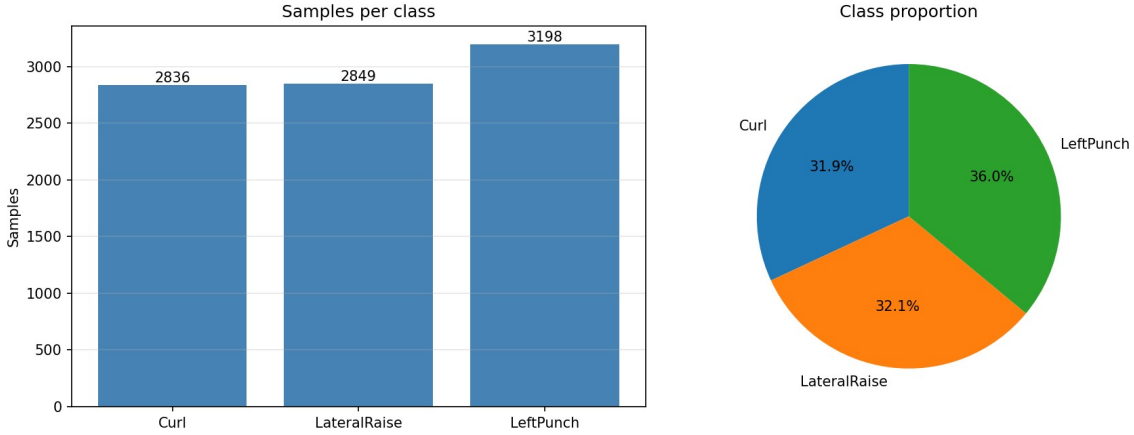


Figure 3. Distribution of raw IMU samples by activity.

The full six-axis time series for each exercise are in docs/task1/timeseries_Curl.png, timeseries_LateralRaise.png, and timeseries_LeftPunch.png.

Task 1 Six Axis Statistics and Sampling

Table 3. Raw IMU mean, standard deviation, minimum and maximum by class

Activity	Channel	Mean	SD	Min	Max
Curl	acc_x	-0.129	0.133	-0.675	0.354
Curl	acc_y	0.278	0.233	-0.597	0.878
Curl	acc_z	0.792	0.746	-1.754	2.259
Curl	gyro_x	-0.343	157.327	-279.160	264.530
Curl	gyro_y	-1.818	47.405	-169.190	160.090
Curl	gyro_z	-1.702	28.144	-134.750	104.300
Lateral raise	acc_x	-0.227	0.117	-0.543	0.460
Lateral raise	acc_y	-0.877	0.409	-1.627	0.767
Lateral raise	acc_z	-0.445	0.254	-1.702	0.474
Lateral raise	gyro_x	-0.137	79.412	-157.430	132.020
Lateral raise	gyro_y	-1.623	28.979	-113.680	112.280
Lateral raise	gyro_z	1.229	32.286	-70.560	143.220
Left punch	acc_x	-0.542	0.447	-2.190	0.518
Left punch	acc_y	-0.089	0.519	-1.893	1.833
Left punch	acc_z	-0.473	0.439	-1.741	1.333
Left punch	gyro_x	0.583	32.699	-146.580	123.760
Left punch	gyro_y	3.025	105.779	-256.340	229.390
Left punch	gyro_z	4.216	66.581	-182.840	191.450

Acceleration is in g and angular rate is in degrees per second. SD is the sample standard deviation.

Table 4. Capture duration and active logging rate

Stream	Activity	Rows	Elapsed s	Active Hz	Gaps >100 ms
Raw	Curl	2,836	38.87	72.94	0
Raw	Lateral raise	2,849	39.07	72.89	0
Raw	Left punch	3,198	291.12	71.5	1
Quat.	Curl	4,044	40.43	100.0	0
Quat.	Lateral raise	4,050	40.49	100.0	0
Quat.	Left punch	4,051	40.5	100.0	0

The raw left-punch wall-clock span is 291.12 s because it includes the discontinuity. Its active logging rate is 71.50 samples/s.

Task 1 Logged Quaternion Statistics and Timing

Table 5. Measured quaternion component statistics by activity

Activity	Component	Mean	SD	Min	Max
Curl	q0	-0.177	0.115	-0.452	0.183
Curl	q1	0.215	0.230	-0.237	0.689
Curl	q2	0.315	0.171	-0.147	0.699
Curl	q3	0.850	0.076	0.643	0.997
Lateral raise	q0	0.171	0.034	0.074	0.257
Lateral raise	q1	-0.429	0.199	-0.775	-0.027
Lateral raise	q2	0.318	0.116	0.022	0.628
Lateral raise	q3	0.790	0.085	0.593	0.953
Left punch	q0	0.601	0.058	0.335	0.733
Left punch	q1	0.056	0.074	-0.124	0.252
Left punch	q2	0.069	0.056	-0.087	0.204
Left punch	q3	0.786	0.042	0.673	0.930

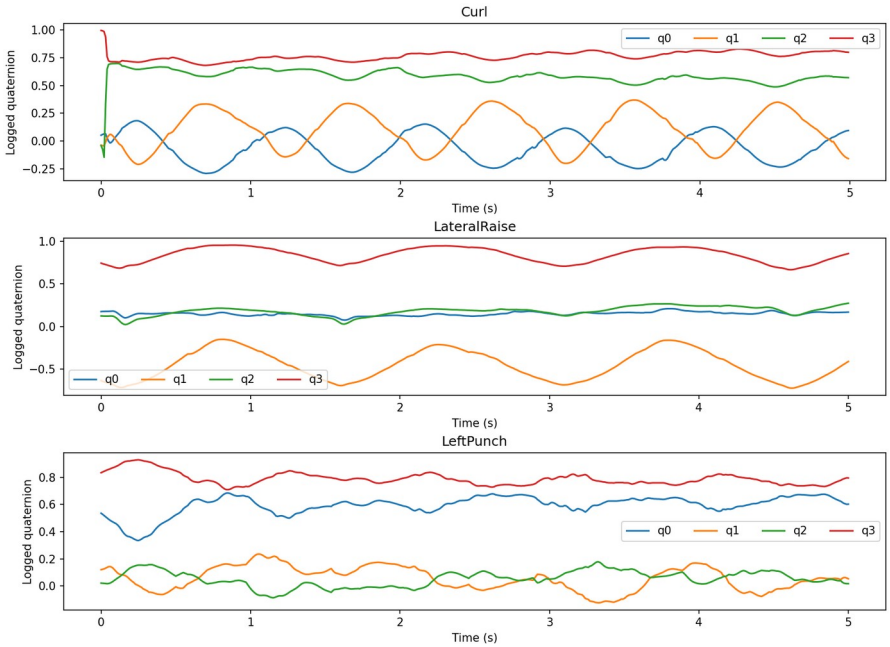


Figure 4. First five seconds of each separately logged quaternion take.

The displayed component names follow the MEMS Studio export. Confirm the scalar/vector convention and align the firmware output with this training order before deployment.

Task 1 Temporal Dynamics

Table 6. Dominant peaks of the IMU magnitude spectra

Activity	Acceleration peak (Hz)	Angular-rate peak (Hz)
Curl	0.78	1.56
Lateral raise	0.78	1.17
Left punch	1.17	1.17

Curl and lateral raise share the 0.78 Hz acceleration-magnitude peak in these recordings. Their angular-rate peaks differ. Left punch shows a 1.17 Hz peak in both magnitude spectra. These are dominant spectral components of the traces. Magnitude spectra can contain harmonics, so a peak need not equal the repetition frequency.

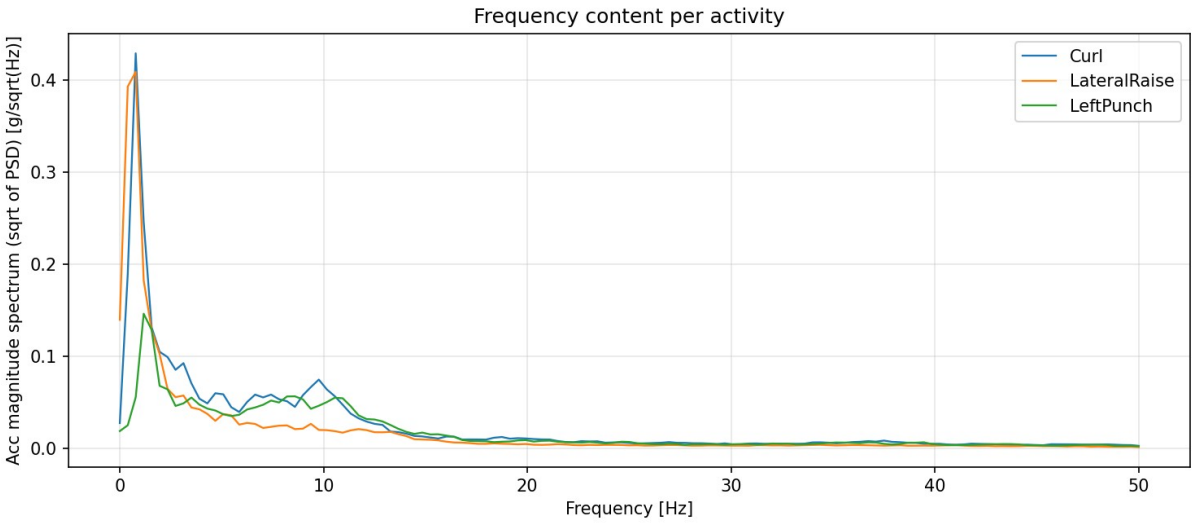


Figure 5. Acceleration-magnitude spectral estimates from continuous raw recording segments.

The classes account for approximately 31.9–36.0% of raw samples each. The separate quaternion takes are nearly balanced at about one third each. This keeps any single class from dominating the reported accuracy.

Task 2 RAW Model

The baseline predicts one activity per sample from ACC_X, ACC_Y, ACC_Z, GYR_X, GYR_Y, and GYR_Z. The model uses native mg and mdps units. The common tree settings for Tasks 2–5 are Gini impurity, no maximum depth, one sample per leaf minimum, a split minimum of two samples, and seed 42. The partitions preserve time order within each take.

Table 7. RAW accuracy and evaluation counts

Partition	Observations	Accuracy
Train	6,217	100.00%
Validation	1,332	98.05%
Test	1,334	97.83%

Table 8. Validation and test metrics by activity

Split	Activity	Precision	Recall	F1	Support
Validation	Curl	0.984	0.991	0.987	425
Validation	Lateral raise	0.993	0.956	0.974	427
Validation	Left punch	0.968	0.994	0.980	480
Test	Curl	0.983	0.972	0.978	426
Test	Lateral raise	0.990	0.972	0.981	428
Test	Left punch	0.963	0.990	0.976	480

The tree classified 1,305 of 1,334 test samples correctly. Of the 29 errors, 12 were curls predicted as left punch. Lateral raise produced 12 errors, split evenly between curl and left punch. Five left-punch samples were assigned to another activity.

Script: src/model_raw.py. Full tree: models/decision_tree_raw.json. The JSON records the tree nodes, thresholds, child indices, feature order, and classes.

Task 2 Errors and Tree Complexity

Table 9. Test confusion counts by true and predicted class

True class	Curl	Lateral raise	Left punch
Curl	414	0	12
Lateral raise	6	416	6
Left punch	1	4	475

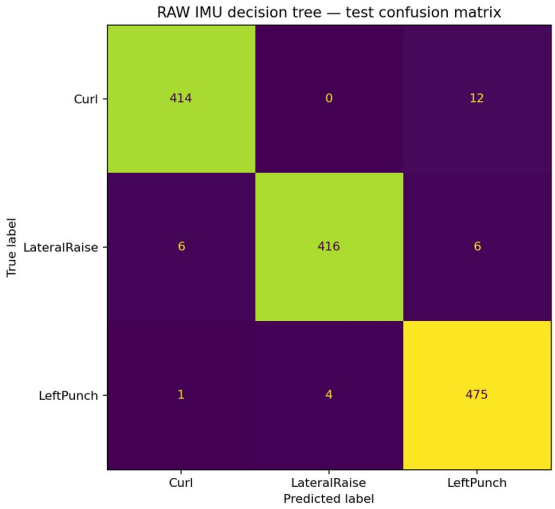


Figure 6. RAW test confusion matrix. The rows are true activities.

Table 10. RAW model structure and latency

Depth	Leaves	Nodes	JSON (KiB)	Tree estimate (µs)
15	83	165	75.590	2.143

Tree traversal is estimated as depth × 12 cycles at 84 MHz. The saved host benchmark reports a median of 293.78 µs per scikit-learn single-row call, which includes the Python interface overhead.

The tree predicts from a single sample and has no feature window delay. Its large node count increases table storage and the number of branch comparisons.

Task 2 Feature Importance and Tree Structure

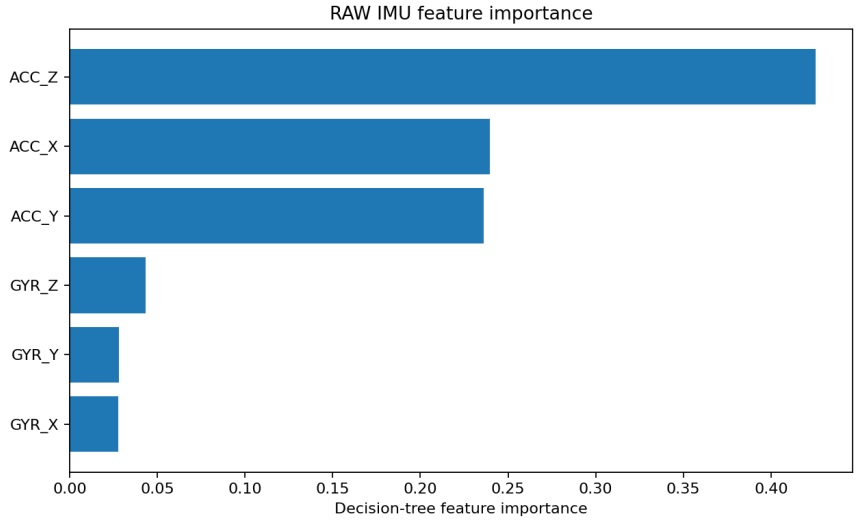


Figure 7. Gini importance of the six raw IMU axes.

ACC_Z had the largest importance (42.5%), followed by ACC_X (24.0%) and ACC_Y (23.6%). The acceleration axes account for about 90% of the total importance for this fitted tree.

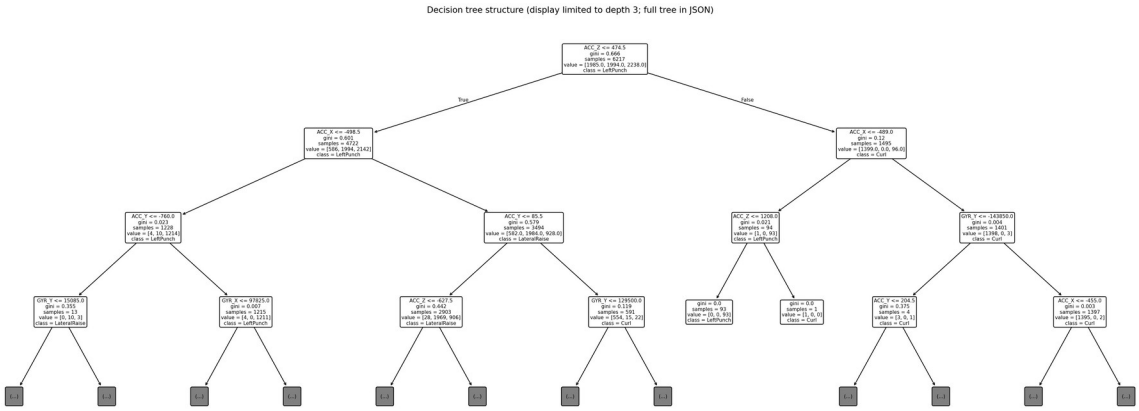


Figure 8. Simplified view of the RAW tree. The full topology is in the exported JSON.

Task 3 RAW Plus Feature Engineering

The feature extractor uses 42 consecutive samples and a stride of 21 samples for 50% overlap. At the observed raw logging rates, a window takes about 0.58–0.59 s to fill and updates about every 0.29 s. Each axis contributes mean, population variance, maximum, minimum, peak-to-peak amplitude, and mean squared energy. Six statistics across six axes produce 36 features. The model uses the Task 2 tree settings.

Windows stay within their chronological partition and within one recording. The window code skips any interval containing a timestamp jump greater than 100 ms or a non-increasing timestamp. The feature extraction and gap checks are highlighted in Appendix A.

Table 11. RAW+FE accuracy and evaluation counts

Partition	Observations	Accuracy
Train	289	100.00%
Validation	59	100.00%
Test	59	100.00%

Table 12. Validation and test metrics by activity

Split	Activity	Precision	Recall	F1	Support
Validation	Curl	1.000	1.000	1.000	19
Validation	Lateral raise	1.000	1.000	1.000	19
Validation	Left punch	1.000	1.000	1.000	21
Test	Curl	1.000	1.000	1.000	19
Test	Lateral raise	1.000	1.000	1.000	19
Test	Left punch	1.000	1.000	1.000	21

Every validation and test window was classified correctly, including all 21 left-punch test windows. RAW and RAW+FE use the same raw recordings, but predict at different granularities. The sample and window accuracy figures therefore describe different decision units.

Script: src/model_raw_fe.py. Full tree: models/decision_tree_raw_fe.json.

Task 3 Classification and Complexity

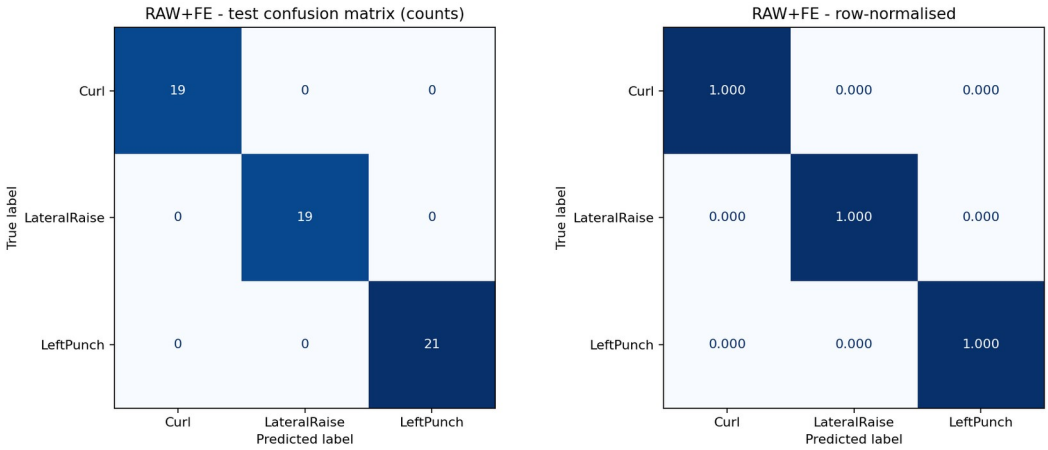


Figure 9. RAW+FE test confusion matrix. The rows are true activities.

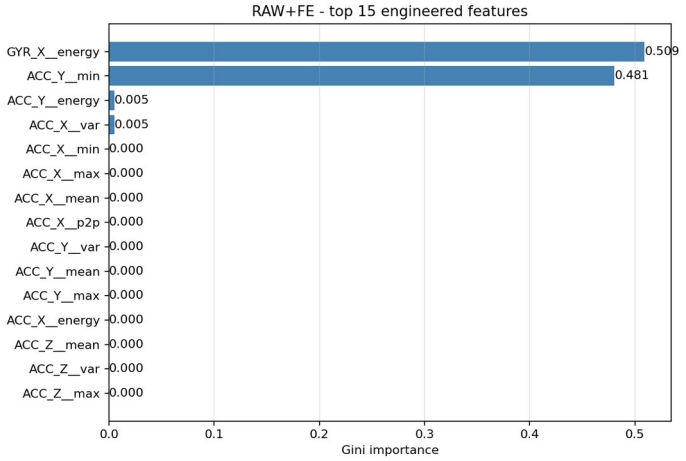


Figure 10. Most important engineered raw-axis features.

Gyroscope X energy (50.9%) and accelerometer Y minimum (48.1%) account for nearly all split importance. The fitted tree uses four of the 36 computed inputs.

Table 13. RAW and RAW+FE implementation trade-offs

Model	Depth	Leaves	JSON (KiB)	Window RAM (B)	Tree estimate (µs)
RAW	15	83	75.590	0	2.143
RAW+FE	3	5	6.517	1008	0.429

Window RAM counts the 42 × 6 float32 input buffer. The 0.429 µs estimate covers tree traversal. Feature extraction averaged 26.65 µs per window in the saved Python host run. The wait for a full window is a separate cost.

Task 4 Quaternion Model

QUAT applies the baseline decision-tree settings to one row of the measured q0–q3 stream. The exported model expects components in the order in the recording. Confirm which logged component is scalar before translating the inputs into the assignment's scalar-first notation on the STM32. The four features are orientation components. The model omits raw linear acceleration and angular velocity as direct inputs.

Table 14. QUAT accuracy and evaluation counts

Partition	Observations	Accuracy
Train	8,500	100.00%
Validation	1,822	97.97%
Test	1,823	87.22%

Table 15. Validation and test metrics by activity

Split	Activity	Precision	Recall	F1	Support
Validation	Curl	0.943	1.000	0.970	607
Validation	Lateral raise	1.000	0.939	0.969	607
Validation	Left punch	1.000	1.000	1.000	608
Test	Curl	0.723	1.000	0.839	607
Test	Lateral raise	1.000	0.617	0.763	608
Test	Left punch	1.000	1.000	1.000	608

QUAT scored 87.22% on the test partition versus 97.83% for RAW. A likely explanation is that a single quaternion describes the sensor's orientation at one instant. Curls, lateral raises, and left punches can pass through similar poses even when their acceleration and angular rate differ. The RAW model has direct access to those motion cues. The raw and quaternion takes were recorded separately, so the accuracy gap also reflects differences between recordings.

Script: src/model_quaternion.py. Full tree: models/decision_tree_quaternion.json. Task 6 shows the confusion matrices for RAW, RAW+FE, and QUAT together with QUAT+FE.

Task 4 Errors Feature Importance and Cost

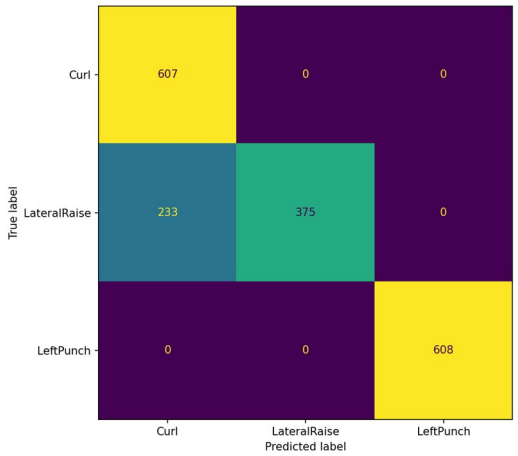


Figure 11. QUAT test confusion matrix. The rows are true activities.

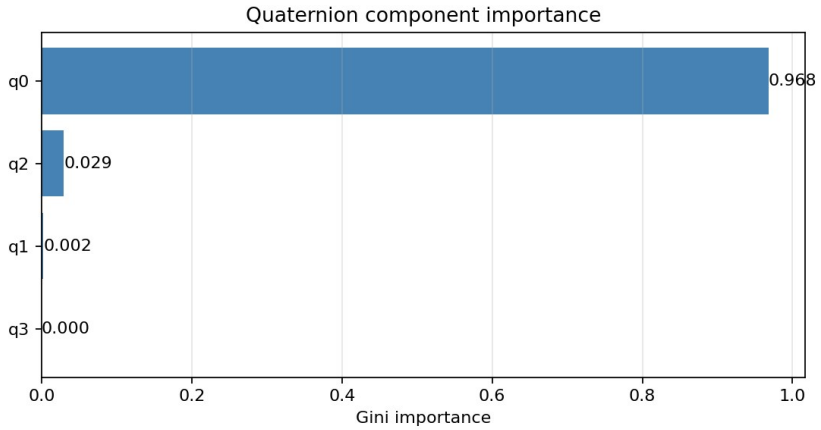


Figure 12. Gini importance by recorded quaternion component.

All 233 errors were lateral raises predicted as curls. Lateral-raise recall fell to 61.68%, while both other classes retained 100% recall. q0 accounts for 96.8% of split importance, followed by q2 at 2.95% and q1 at 0.25%.

QUAT uses nine nodes, five leaves, and depth three. The JSON is 4.003 KiB versus 75.590 KiB for RAW. The depth-based traversal estimate is 0.429 µs. A scikit-learn prediction call averaged 44.97 µs on the host. Quaternion acquisition or fusion cost belongs in the end-to-end timing.

Task 5 Quaternion Feature Engineering and Selection

We computed the same six statistics on 42 × 4 quaternion windows with 21-sample stride. The 100 samples/s stream fills a window in approximately 0.42 s and updates the prediction every 0.21 s. Four channels produce 24 candidates. ANOVA F-scores report individual class separation, and recursive feature elimination (RFE) uses a 200-tree random forest to rank the inputs. The selection is analogous to a feature-screening workflow in MEMS Studio and is performed in Python on training windows.

RFE retained the first eight ranked inputs and eliminated ranks 9–24 as less useful under the fitted forest ranking. This ranking can differ from ANOVA because RFE reevaluates importance after each feature removal. The final classifier is a single decision tree with the same settings as the baseline.

Table 16. All 24 candidate features with ANOVA and RFE ranks

Rank	Candidate	ANOVA F	p-value	Kept
1	q0__mean	5,030.60	3.14e-283	Yes
2	q0__min	6,921.18	6.70e-310	Yes
3	q1__max	1,020.85	1.98e-157	Yes
4	q1__min	838.38	2.00e-143	Yes
5	q0__max	3,930.33	8.16e-263	Yes
6	q1__mean	711.29	3.93e-132	Yes
7	q0__energy	4,136.90	4.93e-267	Yes
8	q2__max	499.17	3.34e-109	Yes
9	q1__p2p	399.08	7.88e-96	No
10	q0__p2p	360.00	5.46e-90	No
11	q2__energy	175.39	2.40e-55	No
12	q1__energy	184.37	2.15e-57	No
13	q0__var	166.44	2.96e-53	No
14	q2__mean	307.37	1.92e-81	No
15	q1__var	235.92	2.68e-68	No
16	q2__min	157.03	5.32e-51	No
17	q3__var	24.61	8.36e-11	No
18	q3__p2p	23.90	1.57e-10	No
19	q3__min	8.76	1.89e-04	No
20	q2__p2p	57.50	1.15e-22	No
21	q3__max	47.41	3.40e-19	No
22	q2__var	42.09	2.61e-17	No
23	q3__mean	26.39	1.73e-11	No
24	q3__energy	28.28	3.29e-12	No

The p-values describe the training windows. Overlapping windows are temporally correlated.

Task 5 Classification and Resource Cost

Table 17. QUAT+FE accuracy and evaluation counts

Partition	Observations	Accuracy
Train	401	100.00%
Validation	81	100.00%
Test	81	100.00%

Table 18. Validation and test metrics by activity

Split	Activity	Precision	Recall	F1	Support
Validation	Curl	1.000	1.000	1.000	27
Validation	Lateral raise	1.000	1.000	1.000	27
Validation	Left punch	1.000	1.000	1.000	27
Test	Curl	1.000	1.000	1.000	27
Test	Lateral raise	1.000	1.000	1.000	27
Test	Left punch	1.000	1.000	1.000	27

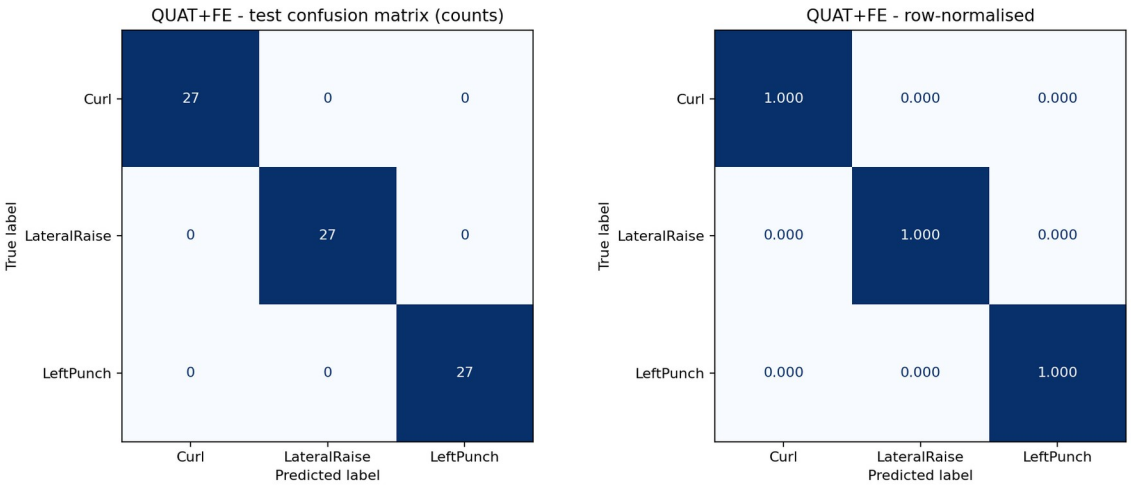


Figure 13. QUAT+FE test confusion matrix. The rows are true activities.

The 81 test windows contain 27 windows per activity. The JSON export is models/decision_tree_quaternion_fe.json. The training and selection script is src/model_quaternion_fe.py.

Task 5 Selected Components and Statistics

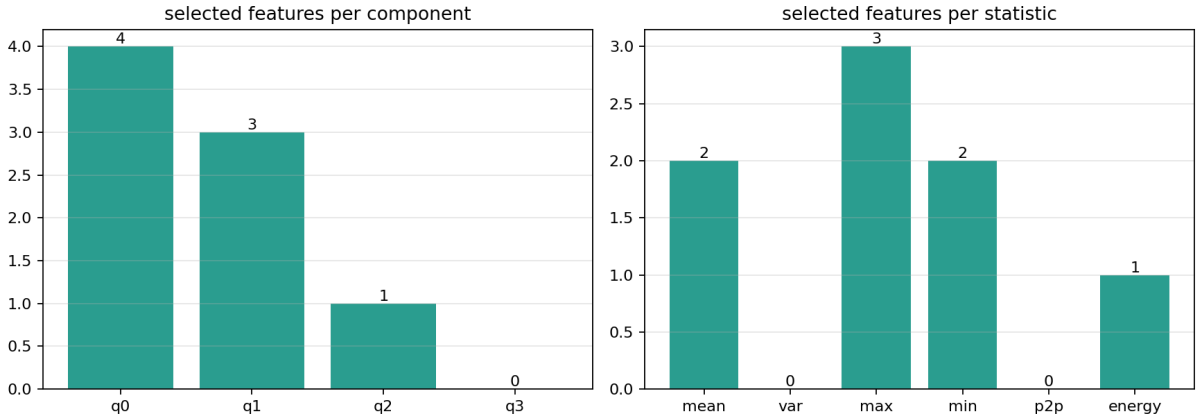


Figure 14. Selected inputs grouped by component and window statistic.

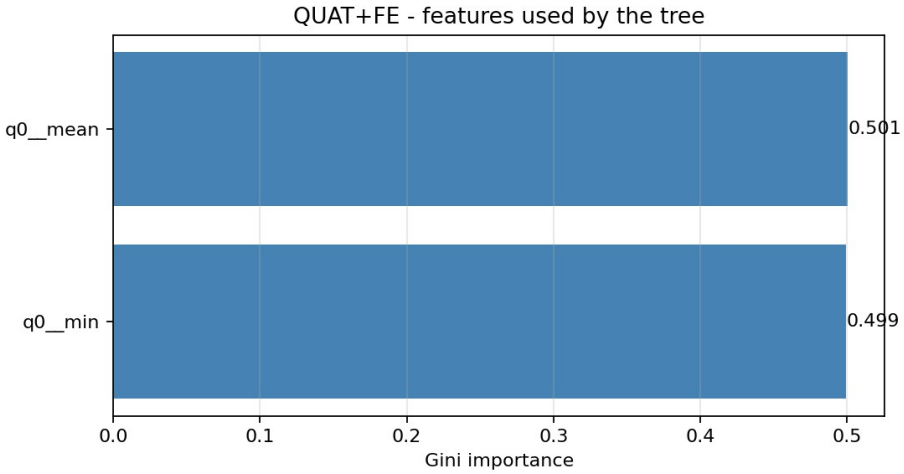


Figure 15. Importance of the eight retained features in the final tree.

RFE retained four features from q0, three from q1, and one from q2. The final tree used only q0 mean and minimum, with split importance of 50.06% and 49.94%. Windowing improved the result to 81 of 81 test windows on these recordings, but both selected features still summarize absolute orientation. This dependence on the recorded poses motivated the window-relative features in Task 7.

Table 19. Window models and the raw quaternion model

Model	Inputs	Depth	Leaves	JSON (KiB)	Window RAM (B)
RAW+FE	36	3	5	6.517	1,008
QUAT	4	3	5	4.003	0
QUAT+FE	8	2	3	4.665	672

The 672 B figure is the float32 42 × 4 input buffer. The generated extractor also uses feature scratch storage. Python feature extraction averaged 28.59 µs per window on the host.

Task 6 Comprehensive Model Comparison

The comparison uses chronological test partitions and reports macro precision, recall, and F1 across the three activities. Single-sample trees make one decision per input row. Window trees use 42 rows per decision and advance by 21 rows. Since each stream comes from a separate take, the table describes the observed results for each pipeline under its own recording conditions.

Table 20. Four-model performance and implementation metrics

Metric	RAW	RAW+FE	QUAT	QUAT+FE
Decision unit	1 sample	42-sample window	1 sample	42-sample window
Test observations	1,334	59	1,823	81
Training accuracy	100.00%	100.00%	100.00%	100.00%
Validation accuracy	98.05%	100.00%	97.97%	100.00%
Test accuracy	97.83%	100.00%	87.22%	100.00%
Macro precision	97.91%	100.00%	90.75%	100.00%
Macro recall	97.78%	100.00%	87.23%	100.00%
Macro F1	97.84%	100.00%	86.73%	100.00%
Train minus test (pp)	2.17	0.00	12.78	0.00
Model inputs	6	36	4	8
Inputs used by tree	6	4	3	2
Tree depth	15	3	3	2
Leaves	83	5	5	3
Nodes	165	9	9	5
JSON (KiB)	75.590	6.517	4.003	4.665
Tree table estimate (B)	3,300	180	180	100
Input window RAM (KiB)	0.000	0.984	0.000	0.656
Tree traversal estimate (µs)	2.143	0.429	0.429	0.286

pp = percentage points. JSON size uses 1 KiB = 1,024 B. The compiled C tree table estimate uses 20 B per aligned node. Window RAM excludes stack and feature scratch space. MCU traversal estimates use depth × 12 cycles / 84 MHz and exclude preprocessing, sensor I/O and serial transport.

The QUAT tree has the smallest serialized model and avoids input window storage. Its lateral-raise errors reduce the test accuracy. QUAT+FE ties RAW+FE for accuracy in these captures while using fewer leaves and a smaller input buffer.

Task 6 Confusion Matrices and Model Cost Charts

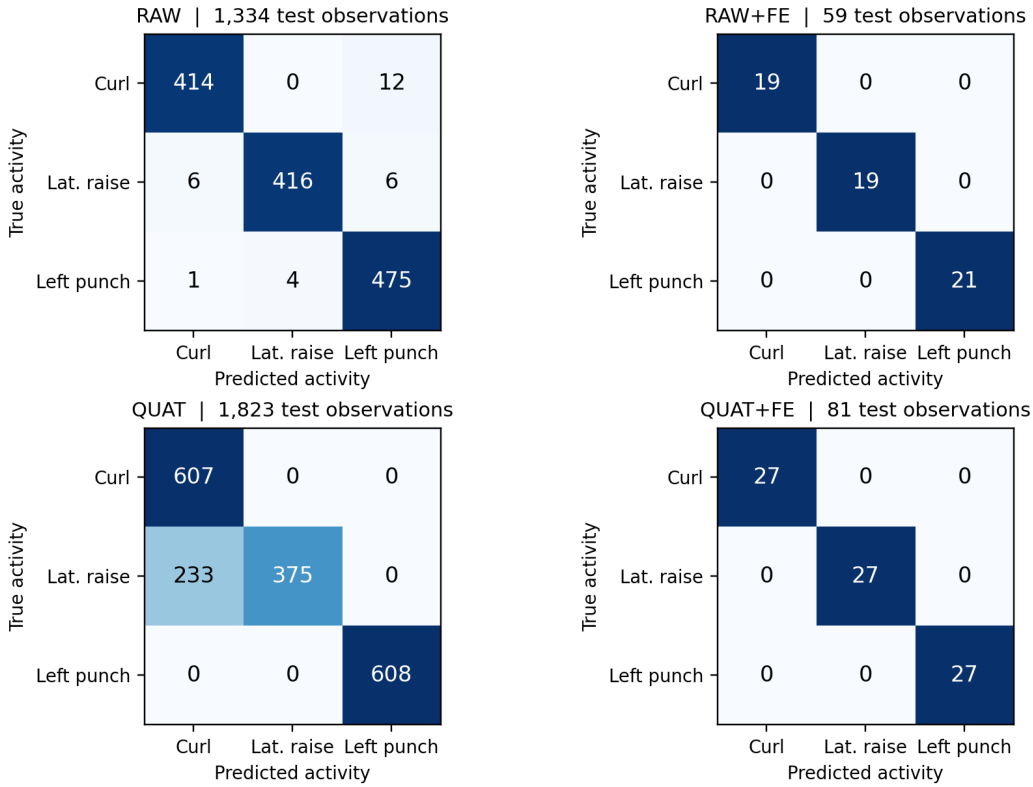


Figure 16. Test confusion matrices for all four models. Colors show proportions within each true class. Labels show counts.

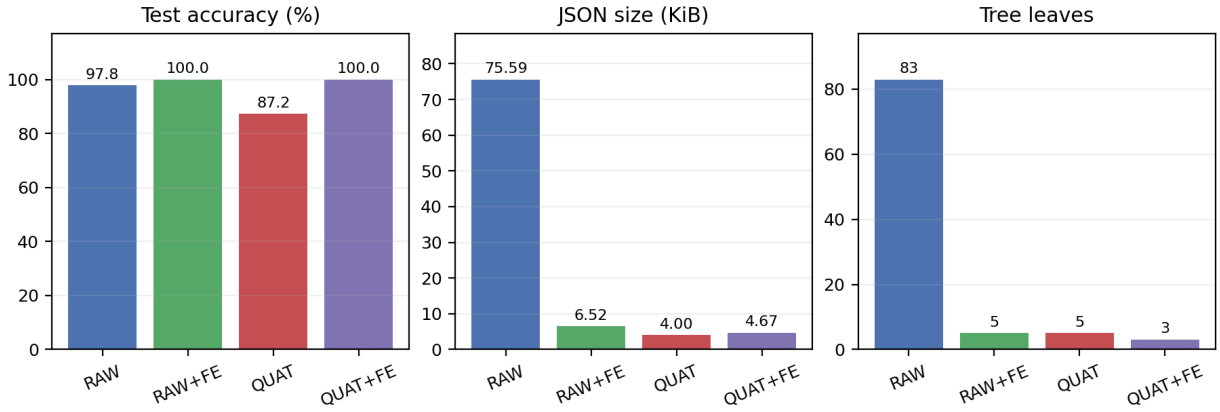


Figure 17. Test accuracy, serialized model size, and leaf count.

RAW+FE and QUAT+FE correctly classify every test window. The biggest class-specific difference is lateral raise, where QUAT predicts curl on 233 of 608 samples. This error accounts for the full QUAT test deficit.

Task 6 Model Ranking and Block Checks

Table 21. Performance rankings with ties

Criterion	RAW	RAW+FE	QUAT	QUAT+FE
Test accuracy	3	1	4	1
Fewest leaves	4	2	2	1
Smallest JSON	4	3	1	2
Smallest tree + window	4	3	1	2
Smallest train/test gap	3	1	4	1

Rank 1 indicates the best result in the row. Equal values receive the same rank.

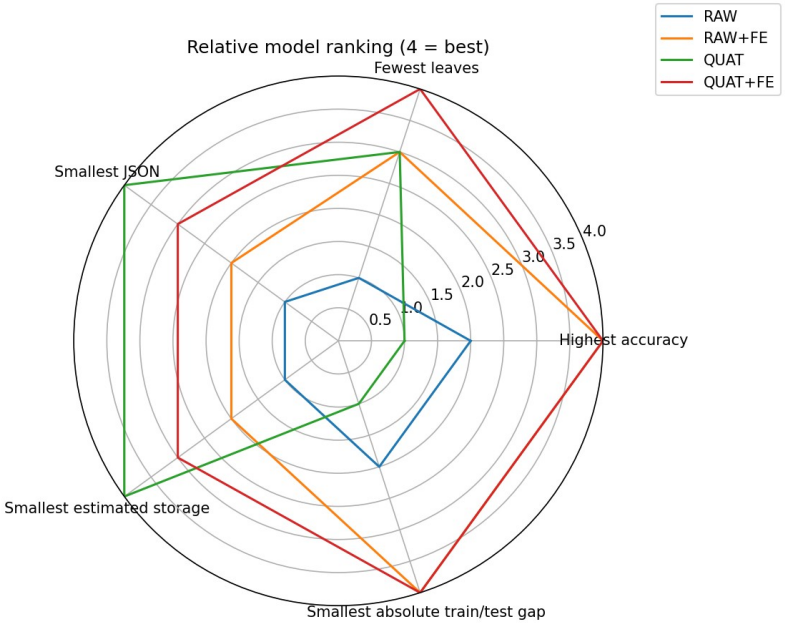


Figure 18. Relative ranks across five criteria. A score of four is the highest.

Table 22. Accuracy across the five time block holdouts

Model	Block 1	Block 2	Block 3	Block 4	Block 5	Mean	Worst
RAW	93.1%	97.4%	97.9%	97.1%	97.5%	96.6%	93.1%
RAW+FE	98.7%	98.8%	97.5%	97.5%	100.0%	98.5%	97.5%
QUAT	93.6%	100.0%	100.0%	100.0%	89.7%	96.7%	89.7%
QUAT+FE	98.2%	100.0%	100.0%	100.0%	100.0%	99.6%	98.2%

Each fold holds out one contiguous block per recording and fits the tree on the other four. QUAT+FE repeats RFE inside each fold using the training blocks. QUAT+FE reached 99.64% mean accuracy and RAW+FE reached 98.51%. These time-block checks reuse the same recording sessions.

Task 6 Model Choice and STM32 Deployment

Table 23. Decision matrix for the observed exercises

Use case	Candidate	Evidence and implementation trade-off
Highest observed test accuracy	RAW+FE / QUAT+FE	Both reached 100%. QUAT+FE had the smaller input buffer and fewer leaves.
Compact sample-level orientation	QUAT	Smallest JSON at 4.003 KiB and no window buffer. Lateral-raise recall was 61.68%.
Immediate prediction from six axes	RAW	97.83% test accuracy with one decision per sample. The tree has 83 leaves.
Candidate for embedded window deployment	QUAT+FE	Three leaves and a 672 B input buffer. Budget for quaternion generation, feature computation and window fill.

Export and host validation

The QUAT export implements traversal in decision_tree_quaternion.c and declares the API in decision_tree_quaternion.h. We compiled the export with GCC on the host and replayed all 1,823 chronological QUAT test inputs. The C result agreed with the saved Python prediction on all 1,823 rows. This checks the exported decision logic and class IDs on the supplied test inputs.

Table 24. STM32F401RE QUAT implementation and host replay

Item	Implementation or evidence
Inputs	Supply q0–q3 in the order in the training CSV. Confirm the scalar/vector mapping of the target stream.
Inference call	quat_raw_tree_predict(q). Invoke once per fresh quaternion.
Class mapping	0 = curl, 1 = lateral raise, 2 = left punch, 255 = unknown.
Host replay agreement	1,823 / 1,823. GCC-compiled C compared with saved Python predictions.

Task 7 ISPU Implementation

We implemented a six-axis Madgwick filter and a decision tree for curls, lateral raises, and left punches on the LSM6DSO16IS ISPU. The sensor hub on the NUCLEO-F401RE is the host interface. The ISPU code computes the quaternion, extracts features, and produces the activity ID. [8]

Table 25. Task 7 sensor and inference configuration

Parameter	Setting
Accelerometer	±8 g, high-performance mode, 208 Hz
Gyroscope	±2,000°/s, high-performance mode, 208 Hz
ISPU processing	208 Hz, with gyro bias calibration before classification
Quaternion rates	120 frames/s nominal selection for classification. The host logs averaged 120.48 rows/s.
Activity IDs	0 = curl, 1 = lateral raise, 2 = left punch. 255 = startup or window not ready.
Window and stride	42 samples per window and 21 samples per step. Nominal intervals are 350 ms and 175 ms.

At startup, the fusion code estimates gyroscope bias from about two seconds of stationary data and initializes orientation from gravity. It then updates the quaternion each native sensor tick. A phase accumulator selects frames for the classifier without interpolation. The frame buffer uses scalar-first order (w, x, y, z). The register map exports x, y, z, w, which MEMS Studio logs as q1, q2, q3, q4. [8]

ISPU rate selection and output

```
#define ISPU_HZ 208u
#define QUAT_OUTPUT_HZ 120u
...
resample_phase+=QUAT_OUTPUT_HZ;
if(resample_phase<ISPU_HZ) return;
resample_phase-=ISPU_HZ;
...
cast_uint32_t(ISPU_DOUT_20)=activity_id;
cast_uint32_t(ISPU_DOUT_22)=prediction_count;
cast_float(ISPU_DOUT_28)=leaf_training_purity;
```

Source excerpt from ISPU_208hz/src/main.c. ISPU_208hz/conf.txt defines the sensor ranges and enables algorithm 0 and INT2. ISPU_208hz/meta.txt defines the host-visible outputs. [8]

Task 7 Relative Features and C Tree

The feature extractor expresses each orientation relative to the first sample in its window. For a unit quaternion q_0 at the window start and a later sample q_i, the relative rotation is $\text{conj}(q_0) \times q_i$. The extractor also computes squared quaternion differences between consecutive samples and from the first sample. These inputs capture how orientation changes during the exercise. A fixed global rotation applied to the whole window cancels from the relative rotation and preserves the squared differences. [8]

Table 26. Relative quaternion feature and tree design

Component	Implementation
Feature channels	Four relative quaternion components, squared step change, and squared change from the window start
Candidate statistics	Mean, population variance, maximum, minimum, peak-to-peak, and mean-square energy. Six statistics per channel give 36 candidates.
Features used	Eight inputs appear in tree splits. Relative x peak-to-peak and step-change peak-to-peak contribute 48.08% and 46.51% of split importance, respectively.
Tree structure	19 nodes, 10 leaves, depth 4. Each branch compares one feature with a threshold and follows a child index.
Prediction output	Class ID, prediction counter, and leaf training purity. Purity is the fraction of leaf training samples in the predicted class.

Ordered ring buffer and inference

```
q_write=(q_write+1u)%QUATFE_WINDOW;
++q_count;
if(q_count<QUATFE_WINDOW) return;
if(((q_count-QUATFE_WINDOW)%QUATFE_STRIDE)!=0u) return;

float features[QUATFE_FEATURES];
quatfe_window_features(qwin, q_write, features);
QuatFePrediction p=quatfe_predict_features(features);
activity_id=p.predicted_class;
```

Source excerpt from ISPU_208hz/src/main.c. Once the ring buffer is full, q_write points to the oldest sample. The extractor needs that index to preserve time order and anchor the window correctly. quat_fe_tree.c stores the generated C node table. [8]

Task 7 Recordings and Offline Evaluation

We used one LSM6DSO16IS quaternion recording per exercise. The recording filenames supplied the activity labels. We excluded the Activity Number column because it contained predictions from an earlier model. Each recording was divided chronologically into 60% training, a 5% gap, 15% validation, a 5% gap, and 15% test. We rejected eight windows containing a quaternion whose norm deviated from 1 by more than 5%. [8]

Table 27. Task 7 quaternion recordings

Activity	Samples	Duration (s)	Logged rate (Hz)
Curl	4,875	40.454	120.48
Lateral raise	4,881	40.504	120.48
Left punch	4,878	40.479	120.48

Table 28. Relative quaternion model accuracy

Partition	Correct / windows	Accuracy
Train	405 / 410	98.78%
Validation	98 / 100	98.00%
Test	102 / 105	97.14%

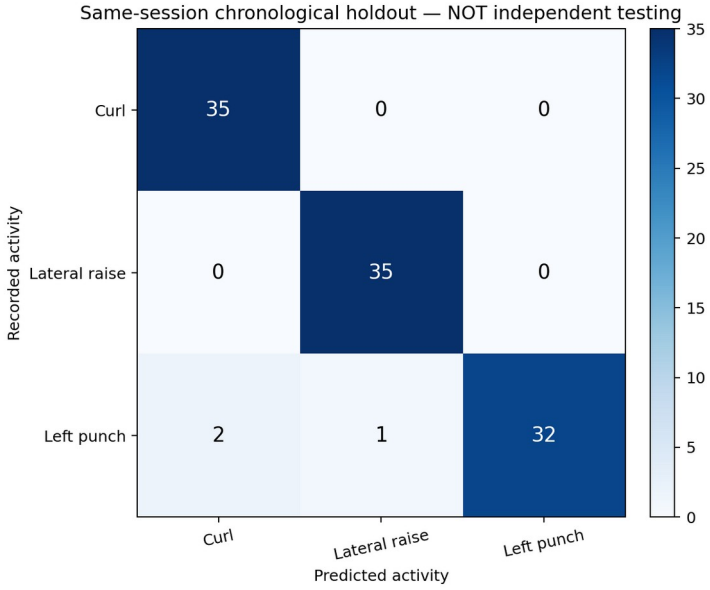


Figure 19. Task 7 test confusion matrix. Rows are true activities and columns are model predictions.

All 35 curls and 35 lateral raises were classified correctly. Of the 35 left-punch windows, 32 were correct, two were predicted as curls, and one was predicted as a lateral raise. The test windows came from held-out portions of the same recording sessions used to develop the model.

Task 7 Verification and GitHub Results

We compiled the exported C feature extractor and decision tree with desktop GCC and replayed the recorded windows. The test compared C features with saved Python features and C class IDs with the Python/JSON reference. The model achieved 100% reference agreement across 615 recorded windows. [8]

Table 29. Desktop implementation verification

Check	Observed result
Predictions on recorded windows	615/615 matched the JSON reference.
Ring-buffer wrap checks	658 checks with zero feature mismatches outside tolerance.
Fixed global rotation test	All 123 checked windows retained their predicted class.
Maximum absolute feature difference	4.17×10^{-7} between desktop C and saved Python features.

Implementation footprint

The 42 × 4 float32 input ring occupies 672 B. The feature vector has 36 float32 entries and occupies 144 B. The 19-node table occupies 380 B in the desktop GCC object. These figures describe the named data structures. The 175 ms decision interval follows from the configured stride and frame rate.

STM32 interface and results

The AlgoBuilder Sensor Hub interface reads the quaternion and activity fields exposed by the ISPU metadata. The package also includes a HAL helper that formats the class ID, prediction counter, and calibration status for a STM32 UART configured at 921600 baud. [8]

Task 7 results are available in the assignment's GitHub repository. GitHub assignment folder

The project package includes the evaluation summary, held-out predictions, confusion matrix, tree plot, and host validation output. The Python script src/retrain_task7.py regenerates the trained model and C tree. The verification script is tests/verify_c_matches_python.py. [8]

Task 7 Memory Output Timing and Sensor Power

Earlier linked firmware footprint

The earlier Task 7 build provides a hardware footprint reference. The ISPU figures come from STRED size output, and the STM32 figures come from STM32CubeIDE Build Analyzer. Both describe the earlier linked firmware images. The revised exercise implementation’s buffer and tree storage figures appear in the preceding section. [9]

Table 30. Earlier linked firmware allocation and device capacity

Target	Program allocation	Static data allocation	Capacity used
LSM6DSO16IS ISPU	21,932 B (21.42 KiB)	108 B data + 784 B BSS = 892 B (0.871 KiB)	66.93% of 32 KiB program RAM and 10.89% of 8 KiB data RAM
STM32F401RE host	80.61 KiB flash	35.70 KiB SRAM	15.74% of 512 KiB flash and 37.19% of 96 KiB SRAM

The ISPU text, data, and BSS sizes sum to 22,824 B. Capacity denominators are from the ST datasheets. [3, 4, 9]

Sampling cadence and window timing

Table 31. Configured intervals and observed log spacing

Quantity	Interval	Basis
Native sensor update	4.808 ms	1 ÷ 208 Hz
Selected quaternion output	8.333 ms nominal	1 ÷ 120 Hz
Revised exercise logs	8.300 ms average	1 ÷ 120.48 Hz, measured log rate [8]
Earlier demonstration log	8.299 ms observed	Difference between two displayed timestamps [10]
Window buffer fill	350 ms nominal	42 samples ÷ 120 Hz
Prediction interval	175 ms nominal	21 samples ÷ 120 Hz

The first four rows describe sensor updates or output cadence. The last two describe window fill and prediction spacing. Inference execution time is a separate quantity.

Datasheet power estimate

ST specifies a typical current of 595 µA for the accelerometer and gyroscope in high-performance mode. At the typical 1.8 V supply, P = VI gives 1.071 mW. This estimate covers the two sensing elements. ISPU and STM32 consumption are outside its scope. [3]

4 Dependent Tasks

Task 1 established the recording streams and their split procedures. Tasks 2–5 change the feature representation and retain the same activity labels and tree settings. Task 6 compares those four trained models using the recorded metrics and replays the exported QUAT tree in C. Task 7 retains the three activity labels and retrains the window model on the LSM6DSO16IS stream using changes in orientation within each window.

Table 32. Dependencies and observed outcomes

Task transition	Change and result
1 to 2	Use chronological raw rows and six axes for the baseline. RAW reached 97.83%.
2 to 3	Add 36 window statistics on the raw stream. The tree went from 83 to five leaves and correctly classified 59 test windows.
2 to 4	Use four recorded quaternion components from separate takes. QUAT reduced JSON size to 4.003 KiB and reached 87.22%.
4 to 5	Window the quaternion stream, rank 24 candidate features and retain eight. QUAT+FE classified 81/81 windows with a three-leaf tree.
2–5 to 6	Aggregate metrics and compare trade-offs. The exported QUAT C tree matched all saved Python test predictions on the host.
5 to 7	Use window-relative quaternion dynamics on the ISPU stream. The model classified 102 of 105 held-out windows correctly.

5 Conclusion

The exercise revision produced four trained decision trees with JSON and C exports. RAW+FE and QUAT+FE led the chronological window tests. QUAT+FE used fewer leaves and less input buffer RAM, while QUAT remained the smallest serialized model. Its lateral-raise errors show the cost of relying on a single orientation sample in this capture.

Task 7 applied relative quaternion features to the same exercises on the LSM6DSO16IS. The resulting tree reached 97.14% accuracy on the chronological holdout, and the desktop C replay matched all 615 reference predictions.

6 References

[1] STMicroelectronics, UM1724, STM32 Nucleo 64 boards MB1136, user manual, Arduino pin mapping and ST-LINK virtual COM interface. https://www.st.com/resource/en/user_manual/um1724-stm32-nucleo64-boards-mb1136-stmicroelectronics.pdf

[2] STMicroelectronics, UM3239, Getting started with the X-NUCLEO-IKS4A1 motion MEMS and environmental sensor expansion board for STM32 Nucleo, Rev. 5, 2025, Sections 3 and 4. https://www.st.com/resource/en/user_manual/um3239-getting-started-with-the-xnucleoiks4a1-motion-mems-and-environmental-sensor-expansion-board-for-stm32-nucleo-stmicroelectronics.pdf

[3] STMicroelectronics, DS13892, LSM6DSO16IS datasheet, Rev. 2, ISPU memory organization and Table 3 electrical characteristics. https://www.st.com/resource/en/datasheet/lsm6dso16is.pdf

[4] STMicroelectronics, STM32F401xD STM32F401xE datasheet, program and SRAM capacities. https://www.st.com/resource/en/datasheet/stm32f401re.pdf

[5] STMicroelectronics, LSM6DSV16X datasheet and AN5763 application note; MEMS Studio 2.4.1 Sensor Evaluation and AlgoBuilder software. https://www.st.com/en/mems-and-sensors/lsm6dsv16x.html

[6] F. Pedregosa et al., Scikit-learn Machine Learning in Python, Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Software used: scikit-learn, NumPy, pandas, and Matplotlib.

[7] DA2 Exercise Revision source archive. Raw and quaternion CSV exports for curl, lateral raise, and left punch, model JSON and C exports, Python scripts in src, and generated tables and plots in docs.

[8] DA2 Task 7 exercise implementation and results, Task7_Curl_LateralRaise_LeftPunch. Project sources, trained JSON model, sensor export CSVs, and offline validation results. GitHub assignment repository

[9] DA2 Task 7 Final Evidence Report, earlier firmware build. STRED section-size output, and STM32CubeIDE Build Analyzer.

[10] Task 7 ISPU Implementation Report, earlier demonstration. MEMS Studio output table and timestamp analysis.

Appendix A Code and Implementation Changes

These excerpts identify the relevant changes and include the surrounding implementation. Yellow highlights mark the feature, split, filtering, and evaluation logic. The archive contains the full scripts.

Task 1 Class Mapping and Stratified Split

```
LABEL_MAP = {"curl": "Curl", "lateralraise": "LateralRaise", "leftpunch": "LeftPunch"}
def stratified_split(data):
    train, temp = train_test_split(
        data, test_size=VAL_FRAC + TEST_FRAC,
        stratify=data["label"], random_state=RANDOM_STATE)
    val, test = train_test_split(
        temp, test_size=TEST_FRAC / (VAL_FRAC + TEST_FRAC),
        stratify=temp["label"], random_state=RANDOM_STATE)
```

The filename mapping assigns the exercise classes consistently. Stratification preserves their proportions during both stages of the split.

Task 2 Chronological Split and Baseline Tree

```
    n = len(frame)
    train_end = int(n * train_fraction)
    validation_end = int(n * (train_fraction + validation_fraction))
    model = DecisionTreeClassifier(
        criterion=args.criterion,
        max_depth=args.max_depth,
        min_samples_leaf=args.min_samples_leaf,
        min_samples_split=args.min_samples_split,
        random_state=args.random_state,
    )
    model.fit(X_train, y_train)
```

The baseline retains the configured tree settings and uses six raw IMU axes. The model scripts split each take into chronological portions before fitting.

Task 3 Window Statistics and Gap Rejection

```
    x = np.asarray(window, dtype=np.float64)
    mean = x.mean(axis=0)
    var = x.var(axis=0)
    mx, mn = x.max(axis=0), x.min(axis=0)
    energy = (x * x).mean(axis=0)
    gaps = np.array([], dtype=int)
    if "time_s" in frame:
        steps = np.diff(frame["time_s"].to_numpy())
        gaps = np.flatnonzero((steps > 0.1) | (steps <= 0))
    for start in range(0, len(frame) - window_size + 1, stride):
        if np.any((gaps >= start) & (gaps < start + window_size - 1)):
            continue
        feats = extract_window_features(raw[start:start + window_size])
```

The window code computes six statistics per axis and rejects windows that span a discontinuity in the timestamps.

Appendix A Quaternion Models and Feature Selection

Task 4 Quaternion Input Cleanup

```
FEATURES=['q0','q1','q2','q3']; CLASSES=['Curl','LateralRaise','LeftPunch']
    found=set(); chosen=[]
    for col in df.columns:
        base=re.sub(r'\.\d+$','',col).strip()
        if base in FEATURES and base not in found: found.add(base);
chosen.append((col,base))
    missing=[f for f in FEATURES if f not in found]
    out=pd.DataFrame({base:pd.to_numeric(df[src],errors='coerce') for src,base in chosen})
    out=out.replace([np.inf,-
np.inf],np.nan).dropna(subset=FEATURES).reset_index(drop=True)
    out = out.loc[(out[FEATURES] ** 2).sum(axis=1) > 0].reset_index(drop=True)
```

The parser takes the first export of each quaternion component, removes invalid zero-norm inputs, and keeps the recorded order for the exported model.

Task 5 Training Feature Selection

```
def anova_table(X, y):
    F, p = f_classif(X, y)
    return pd.DataFrame({"feature": ALL_FEATURES, "anova_F": F, "p_value": p})


def rfe_rank_all(X, y, seed):
    sel = RFE(
        RandomForestClassifier(n_estimators=200, random_state=seed),
        n_features_to_select=1, step=1,
    )
    sel.fit(X, y)
    return pd.Series(sel.ranking_, index=ALL_FEATURES)
        Xtr, ytr = tr[ALL_FEATURES].to_numpy(float), tr.label.astype(str).to_numpy()

        # ---- feature selection (training windows only)
        anova = anova_table(Xtr, ytr)
        rank = rfe_rank_all(Xtr, ytr, args.random_state)
        selected = list(rank.sort_values().index[:args.n_features])
        sel_idx = [ALL_FEATURES.index(f) for f in selected]
```

ANOVA and random-forest RFE run on the training windows. The held-out windows use the same selected columns for prediction.

Appendix A Comparison and Target Call

Task 6 Read the Separate Sensor Streams

```
def load_recordings(paths, quat_paths):
    imu_rec = [(lab, imu.load_recording(Path(p), lab, None)) for lab, p in paths]
    q_rec = [(lab, qf.load_recording(Path(p), lab)) for lab, p in quat_paths]
    return imu_rec, q_rec
```

Task 6 Fit Selection Within Each Holdout

```
        rank = qf.rfe_rank_all(wq_tr[qf.ALL_FEATURES].to_numpy(float),
wq_tr["label"].astype(str).to_numpy(), 42)
        sel = list(rank.sort_values().index[:K_SELECT])         # selection on the
training blocks only
        p = dt().fit(wq_tr[sel], wq_tr["label"]).predict(wq_te[sel])
        res["QUAT+FE"].append(float(accuracy_score(wq_te["label"], p)))
```

Each representation loads its own take. The holdout loop fits the feature selector on the retained training blocks so the held-out block does not determine the selected features.

STM32 Application Call

```
#include "decision_tree_quaternion.h"

/* q_logged_order is a fresh quaternion in the training CSV order. */
quat_raw_tree_class_t activity = quat_raw_tree_predict(q_logged_order);
```

The call is an integration example. The board application supplies the sensor readout, component mapping, output handling, and timing instrumentation.